6CM504 Concurrency and communication

Understanding and modelling concurrency

Prof. Alistair A. McEwan

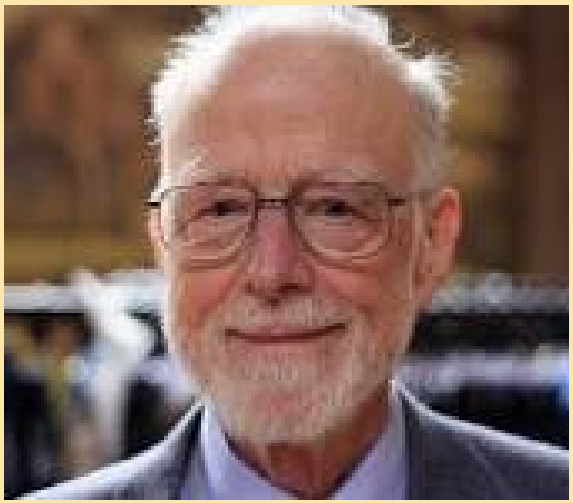
School of Computing
University of Derby

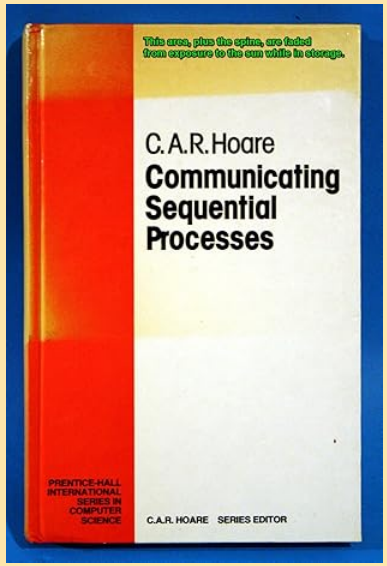
Table of Contents

This week

So far we have seen the basic building blocks of CSP processes, including prefixing, recursion, choice, and channel communication. We have also seen nondeterminism (also known as internal choice). This week, we will use these operators to construct our first models and to show how we can build relatively complex and detailed descriptions of behaviour. Although the models in this seminar are hypothetical examples, they help us understand how models may be constructed, and they will give us some examples to test in the FDR tool.

Table of Contents





- Professor Sir Tony Hoare
- ACM Turing Award winner

Bringing it all together: our first model!

"Forget for a while about computers and computer programming, and think instead about objects in the world around us which act and interact with us and with each other in accordance with some characteristic pattern of behaviour.

Think of clocks and counters and telephones and board games and vending machines.

To describe their patterns of behaviour first decide what kinds of event or action will be of interest."

C. A. R. (Tony) Hoare, Communicating Sequential Processes

Bringing it all together: our first model!

"Think of cars and engine management systems, think of aeroplanes, think of microcontrollers with pre-emptive task sets. Think of safety-critical systems!

"Think of microprocessors with shared memory, think of distributed systems, think of GPUs, think of reconfigurable hardware. Think of programming languages such as Erlang, or Java, or CUDA! To describe their patterns of behaviour first decide what kinds of event or action will be of interest."

6CM504 students

A simple vending machine

- For our first model, we shall construct a simple vending machine
- Firstly, we must decide on our alphabet (the events it can engage in)
- Our vending machine is very simple—there are only two things it can do
  - `coin`: the insertion of a coin
  - `choc`: the extraction of a chocolate from the dispenser
- Question: what is the alphabet of our simple vending machine?

A simple vending machine

```
channel coin, choc

VMEvents = {coin, choc}
```

A simple vending machine

- We can construct a simple vending machine that consumes a coin before breaking

```
BrokenVM = coin -> STOP
```

- This machine offers the event `coin` to its environment
- Once the event has occured, it blocks

A two-shot vending machine

- A better vending machine would be one that is able to serve two customers before breaking

```
BetterVM =
  coin ->
    choc ->
      coin ->
        choc -> STOP
```

- Initially this machine will accept the insertion of a coin, but will not allow a chocolate to be extracted
- After the first coin is inserted, the coin slot closes until the chocolate is extracted
- However it will not accept two coins in a row, nor will it dispense two chocolates in a row
- After serving two chocolates, it breaks

A vending machine with options

- An even better vending machine would be one which offers a choice of a chocolate or a toffee
- Firstly, we must expand out alphabet to include toffees

```
channel toffee
OptionVMEvents = {coin, choc, toffee}
```

- Then we may define our process

```
OptionVM =
  coin -> (
    choc -> OptionVM
    []
    toffee -> OptionVM )
```

A vending machine with options

- There are some important things to note about this vending machine
  - It never breaks—the process is infinitely recusive
  - It insists that exactly one coin is entered for each product extracted. We can observe this by inspection, and it is a property that our vending machine company insists on
  - The choice of a chocolate or a toffee is made by the customer
  - Question: can you explain why this is the case?

An unpopular vending machine

- Assume, due to a manufacturing error we actually delivered the vending machine below
- The vending machine company, and the customers, would not be happy with this vending machine

```
BadOptionVM =
  coin -> (
    choc -> BadOptionVM
    |~|
    toffee -> BadOptionVM )
```

- Question: can you explain why this is the case?

Be careful what you wish for...

- The `OptionVM` and `BadOptionVM` were defined as recursive processes
- Processes can be mutually recusive too

```
PeculiarVM =
  coin -> (
    choc -> ( BadOptionVM |~| OptionVM )
    |~|
    toffee -> OptionVM )
```

- Question: what is peculiar about this vending machine?

A more complicated vending machine

- Our more complicated vending machine offers a choice of coins that may be inserted, as well as a choice of goods and change

```
ComplicatedVMEvents =
  { in100p, in50p, largeChoc,
    smallChoc, out50p }
```

A more complicated vending machine

- Our more complicated vending machine offers a choice of coins that may be inserted, as well as a choice of goods and change

```
CompVM =
  in100p -> ( largeChoc -> CompVM
            []
              smallChoc -> out50p -> CompVM )
  []
  in50p -> ( smallChoc -> CompVM
           []
            in50p -> ( largeChoc -> CompVM
                     []
                       in50p -> STOP ) )
```

The world is sometimes a pragmatic place!

- Our programmers were very ambitious in the design of the complicated vending machine
  - They tried to include lots of functionality
  - In doing so, they didn't fully understand the implications of one of the features they built in
  - Question: what is the design flaw in the complicated vending machine?

The world is sometimes a pragmatic place!

- Our programmers were very ambitious in the design of the complicated vending machine
  - They tried to include lots of functionality
  - In doing so, they didn't fully understand the implications of one of the features they built in
  - Question: what is the design flaw in the complicated vending machine?
- Programmers are pragmatic creatures. On discovering the flaw, they agreed it is easier to change the user manual than to correct the design and so they placed a notice on the machine

"WARNING: Do not insert three 50p coins in a row."

Table of Contents

Summary

- In this lecture we have seen our first models of CSP processes, using the operators we have seen in earlier lectures
- We have seen that we can build relatively sophisticated descriptions of behaviours—and of misbehaviours
- In the coming weeks, we will start to input our models into the FDR tools so that we can reason about them and explore them

Summary

- Advance reading: Schneider pages 467–476
- Advance reading: Roscoe pages 29–35
- Advance reading: Woodcock and Davies, chapter 4